

Reviewer Pack — Discrete Morse Critical Cells of Conflict Complex Bound DPLL...

Entry 02fea1709cc2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 11:47:16 UTC

Discrete Morse Critical Cells of Conflict Complex Bound DPLL Tree Size

Entry ID: 02fea1709cc2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 11:47:16 UTC

1. Conjecture

Field A (mathematical branch): Discrete Morse theory on simplicial complexes (Forman 1998; Joswig-Pfetsch 2005 greedy lex matching; Benedetti-Lutz 2014 random discrete Morse) — a corner of combinatorial topology essentially absent from proof complexity (arXiv ‘discrete Morse’ AND ‘Resolution’ OR ‘DPLL’ OR ‘proof complexity’) returns no direct hits, with <5 adjacent independence-complex papers) **Field B** (complexity object): DPLL / tree-like Resolution refutation tree size $t(F)$ for unsatisfiable 3-CNFs, in the Cook-Reckhow-Krajíček bounded-arithmetic regime where $\neg F$ is a Σ^b_0 statement and $\log_2 t(F)$ controls V^0 -witnessing complexity of unsatisfiability

Statement:

For an unsat 3-CNF F with clauses $c_1, \dots, c_m$, let C_F be the clause-conflict graph (vertex per clause; edge $\{c_i, c_j\}$ iff they share at least one complementary literal pair) and let $I_{\leq 3}(C_F)$ be the independence complex of C_F truncated to faces of size ≤ 4 (faces = clause-subsets pairwise free of complementary-literal conflicts). Run Joswig-Pfetsch greedy lex discrete-Morse matching: traverse simplices in (dim, lex) order, pair each σ with its lex-smallest currently-unmatched coface τ of $\dim(\sigma)+1$ inside the truncation (otherwise mark σ critical). Let $\delta(F) :=$ number of critical cells of dimension ≥ 1 . Conjecture: there is an absolute constant $c \geq 1/4$ such that for every unsat 3-CNF F with $m \geq 4$, $\log_2(t(F)) \geq c \cdot \log_2(\delta(F)+2)$, where $t(F)$ is the DPLL refutation tree-leaf count under lex-variable branching with unit propagation. A single F with measured $\log_2(t^*(F)) < 0.25 \cdot \log_2(\delta(F)+2)$ refutes it.

Rationale (proposer’s reasoning):

Critical cells of $I(C_F)$ count obstructions to extending a locally-consistent partial certificate (a set of mutually non-resolving clauses) to a global one, which is precisely the local-to-global obstruction Ben-Sasson-Wigderson width-size trade-offs quantify for DPLL. A logarithmic relation is the natural scale: t^* grows exponentially in hardness while δ grows exponentially in conflict-topological richness, so log-log alignment is plausible. The bridge avoids algebrization (Forman matching is purely combinatorial — no ring, no polynomial extension), evades natural proofs (the invariant is syntactic on F , not on its truth table), and lies in a regime Razborov/Krajíček bounded-arithmetic witnessing leaves open.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4fa4047e2d483933

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_3cnf(n, seed):
    random.seed(seed)
    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for _ in range(int(4.5 * n)):
        clause_vars = random.sample(variables, 3)
        clause = []
        for var in clause_vars:
            if random.choice([True, False]):
                clause.append(var)
            else:
                clause.append(f'¬{var}')
        clauses.append(clause)
    return clauses

def is_sat(clauses, max_depth=1000):
    if not clauses:
        return True
```

```

if max_depth == 0:
    return False
var = clauses[0][0].replace('¬', '')
for sign in [var, f'¬{var}']:
    new_clauses = []
    for clause in clauses:
        if sign in clause:
            continue
        elif f'¬{sign.replace("¬", "")}' in clause:
            new_clause = [lit for lit in clause if lit != f'¬{sign.replace("¬", "")}']
            if not new_clause:
                break
            new_clauses.append(new_clause)
        else:
            new_clauses.append(clause)
    else:
        if is_sat(new_clauses, max_depth-1):
            return True
return False

def build_conflict_graph(clauses):
    graph = defaultdict(set)
    for i, clause1 in enumerate(clauses):
        for j, clause2 in enumerate(clauses):
            if i >= j:
                continue
            conflict = False
            for lit1 in clause1:
                for lit2 in clause2:
                    if (lit1.startswith('¬') and lit2 == lit1[1:]) or (lit2.startswith('¬') and li
                        conflict = True
                        break
                if conflict:
                    break
            if conflict:
                graph[i].add(j)
                graph[j].add(i)
    return graph

def enumerate_independent_sets(graph, max_size=4):
    independent_sets = []
    for size in range(1, max_size+1):
        for nodes in itertools.combinations(graph.keys(), size):
            independent = True
            for i in range(len(nodes)):
                for j in range(i+1, len(nodes)):
                    if nodes[j] in graph[nodes[i]]:
                        independent = False
                        break
                if not independent:
                    break
            if independent:
                independent_sets.append(set(nodes))
    return independent_sets

def greedy_lex_matching(independent_sets):

```

```

matched = set()
critical_cells = []
for sigma in sorted(independent_sets, key=lambda x: (len(x), sorted(x))):
    if sigma in matched:
        continue
    for tau in sorted(independent_sets, key=lambda x: (len(x), sorted(x))):
        if len(tau) != len(sigma) + 1:
            continue
        if sigma.issubset(tau) and tau not in matched:
            matched.add(tau)
            break
    else:
        critical_cells.append(sigma)
return critical_cells

def dpll(clauses, assignments=None):
    if assignments is None:
        assignments = {}
    if not clauses:
        return 1
    for clause in clauses:
        if all(lit.replace('¬', '') in assignments and (lit.startswith('¬') != assignments[lit.replace('¬', '')]) for lit in clause):
            return 0
    unit_clauses = [clause for clause in clauses if len(clause) == 1]
    if unit_clauses:
        lit = unit_clauses[0][0]
        var = lit.replace('¬', '')
        assignments[var] = not lit.startswith('¬')
        new_clauses = []
        for clause in clauses:
            if lit in clause:
                continue
            elif f'¬{var}' in clause:
                new_clause = [c for c in clause if c != f'¬{var}']
                if not new_clause:
                    return 0
                new_clauses.append(new_clause)
            else:
                new_clauses.append(clause)
        return dpll(new_clauses, assignments)
    var = clauses[0][0].replace('¬', '')
    total = 0
    for val in [True, False]:
        new_assignments = assignments.copy()
        new_assignments[var] = val
        new_clauses = []
        for clause in clauses:
            if var in clause and val:
                continue
            elif f'¬{var}' in clause and not val:
                continue
            new_clause = [lit for lit in clause if lit.replace('¬', '') != var]
            if not new_clause:
                continue
            new_clauses.append(new_clause)
        total += dpll(new_clauses, new_assignments)

```

```

    return total

def run_trial(seed):
    n = random.choice([12, 16, 20, 24, 28])
    clauses = generate_3cnf(n, seed)
    if is_sat(clauses):
        return {
            "metric_name": "log2(t*)/log2(δ+2)",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    graph = build_conflict_graph(clauses)
    independent_sets = enumerate_independent_sets(graph)
    critical_cells = greedy_lex_matching(independent_sets)
    delta = len([cell for cell in critical_cells if len(cell) >= 1])
    t_star = dpll(clauses)
    if t_star == 0:
        return {
            "metric_name": "log2(t*)/log2(δ+2)",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    ratio = math.log2(t_star) / math.log2(delta + 2)
    conjecture_holds = ratio >= 0.25
    counterexample = "" if conjecture_holds else f"log2(t*)/log2(δ+2) = {ratio} < 0.25"
    return {
        "metric_name": "log2(t*)/log2(δ+2)",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)
    metric_values = [r["metric_value"] for r in results if r["metric_value"] != 0.0]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_instances")
        sys.exit(0)
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        counterexamples = [r["counterexample"] for r in results if r["counterexample"]]

```

```

if counterexamples:
    first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
    print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

7}

```

TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
TRIAL: {'metric_name': 'log2(t*)/log2(δ+2)', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture': 'log2(t*)/log2(δ+2)'}
RESULT: INCONCLUSIVE reason=no_valid_instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	414162
2	preregistration	ollama_remote	qwen3:8b	0	0	182409
3	novelty	ollama_remote	qwen3:8b	0	0	153620
4	novelty	ollama_remote	qwen3:8b	0	0	146727
5	test_gen	mistral	codestral-latest	0	0	312001
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	292572
7	test_gen	mistral	codestral-latest	0	0	310842
8	test_gen	mistral	codestral-latest	0	0	315033
9	critic	ollama_remote	qwen3:8b	0	0	250613
10	judge	ollama_remote	qwen3:8b	0	0	202724

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 2580703 ms total latency. Provider mix: {'claude_max': 1, 'ollama_remote': 6, 'mistral': 3}

(full prompt+response transcripts available in research/audit/02fea1709cc2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/02fea1709cc2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/02fea1709cc2.tar.gz (if generated)